

Guión de la práctica IV, programación distribuida con MPICH

Joaquín Cuéllar Padilla joaquin.cuellar@uco.es

Abril de 2026

Resumen

En esta práctica vais a introducir os en la programación distribuida usando la librería MPICH. Vais a programar en C. Los objetivos son:

- Practicar la programación en C. Uso de memoria, depuración...
- Diseñar e implementar un algoritmo de cálculo que distribuya la carga computacional entre distintos nodos, usando programación distribuida.
- Utilizar para ese fin la librería MPICH, saber leer la documentación oficial, las cabeceras de la librería y buscar ayuda y recursos en la red.
- Defender mediante exposición oral el trabajo desarrollado.

1. Guión de la práctica

Entender el concepto de programación distribuida puede costar de primeras, así, lo vamos a plantear mediante ejemplos en forma escalonada hasta llegar al problema que se os pide.

Los pasos a dar en el desarrollo de la práctica serán:

1.1. Analizar los ejemplos en mpitutorial.com

Os recuerdo que el portal se encuentra en: <https://mpitutorial.com>

Nos vamos a parar en los ejemplos:.

Funciones bloqueantes punto a punto. Que nos van a ayudar a probar nuestro clúster en docker, empezar a programar empleando MPICH y la dinámica del paso de mensajes.

Funciones colectivas. Éstas son las importantes para nuestro objetivo, son funciones que permiten trabajar a los nodos de forma colectiva "todos a la vez".

Nos van a permitir repartir datos de forma colectiva, recogerlos e incluso tratarlos al vuelo mientras se recogen.

1.2. Problema avanzado de ejemplo

El algoritmo que vamos a desarrollar en clase, es un algoritmo de cálculo de X números primos.

La resolución de este algoritmo es un problema de prácticas de otros años.

Podéis encontrar el proyecto en https://github.com/gentooza/prime_numbers_MPICH.

Y clonarlo si tenéis git usando el comando:

```
git clone https://github.com/gentooza/prime_numbers_MPICH.git
```

El problema nos ofrece una versión con una sola máquina: *main_one.c* y dos versiones distribuidas *main_v1.c* y *main_v2.c*.

Lo interesante será comparar el temporizado de la versión con una sola máquina respecto a la versión 2 de la aplicación distribuida, y hacernos una serie de preguntas:

- ¿Mejora el cómputo distribuir el cálculo de números primos?
- ¿Es siempre mejor añadir nodos de cómputo?
- ¿Por qué no mejora tanto como cabría esperar la computación distribuida en este caso?
- ¿Se podría mejorar aún más el algoritmo?

1.3. Problema a realizar en esta práctica y que deberéis defender.

¡Vamos a realizar una multiplicación de matrices!

Las matrices serán cuadradas y de un tamaño múltiplo de 8, 1024x1024, 2048x2048, 4096x4096...

Echadle un ojo a [Complejidad computacional de la multiplicación de matrices en wikipedia](#) y especialmente a [multiplicación de matrices en sistemas distribuidos en wikipedia](#).

Las matrices deben ser inicializadas con valores aleatorios entre 1 y 10

La temporización se realizará solo de la computación de la multiplicación, no del cálculo de las matrices iniciales ni de la impresión del resultado.

2. Evaluación de la práctica

Recordemos los objetivos de la práctica:

- Practicar la programación en C.
- Diseñar e implementar un algoritmo de cálculo que distribuya la carga computacional entre distintos nodos, usando programación distribuida.
- Utilizar para ese fin la librería MPICH, saber leer la documentación oficial, las cabeceras de la librería y buscar ayuda y recursos en la red.
- Defender mediante exposición oral el trabajo desarrollado.

2.1. ¿Como se evaluará?

La defensa del trabajo será mediante defensa oral de 20 minutos (30 si sois 3), **que deberá estar debidamente estructurada:**

- **Presentación/ introducción.**
- **Metodología: materiales empleados, organización del grupo de trabajo, algoritmo implementado, herramientas utilizadas, el código, las pruebas a plantear...**
- **Resultados obtenidos, (gráficas se agradecen)**
- **Conclusiones y futuras mejoras.**

Podréis (deberíais) emplear apoyo visual: presentación, capturas, gráficas, código...

2.2. ¿Qué se evaluará?

✓ Quiero que hagáis el trabajo en equipos de 2 ó 3.

✓ Quiero que me habléis de los problemas que encontráis y como los solventáis.

✓ Quiero que seáis capaz de entender una librería por sus cabeceras y por la información y tutoriales que encontréis.

✓ Quiero que me describáis las funciones que empleáis con la intención con que lo hacéis, qué parámetros pasáis que esperáis conseguir.

✓ Quiero que habléis de las pruebas que realizáis y como depuráis la aplicación.

✓ Quiero el código bien estructurado, veo bien separar en funciones cortas, no un solo main() kilométrico.

✓ Quiero que hagáis pruebas de temporizado, **comparando la ejecución en varios supuestos** (siempre con el mismo hardware limitado):

- ejecución en un solo nodo.
- ejecución en 4 nodos.
- ejecución en 4 nodos con dos procesos por nodo.
- ejecución en 8 nodos.
- distintos tipos de tamaño de matriz en cada caso

✓ Qué comentemos los resultados que obtenéis

✗ No quiero que cuando me describáis el código me leáis las cabeceras de las funciones!

✗ No quiero código con comentarios, un código debe de entenderse tal cual.

♥ Me encantaría que usarais git (o mercurial, etc.), aunque igualmente no voy a tener tiempo para revisar los commits de todo el mundo y es algo que no se os puede exigir. (pero deberíais usarlo!)

♥ Me encantaría que os pelearais con [Valgrind](#) para depurar el código o que intentéis otra herramienta y lo compartáis.

♥ Me encantaría que, además, me comentéis como ha sido el proceso de despliegue en un clúster con hardware real.